

Responsive Robot Agents

Event-Triggered Execution, Asynchronous Actions, and Predictive Control

Research Proposal II · Zimo Wen

September 2026

Abstract

A robot agent reasons from observations that are already in the past, while previously issued actions may continue during inference and communication. Target movement, grasp slip, obstruction, or an instruction change can therefore invalidate an otherwise reasonable plan before it finishes executing. This proposal studies responsiveness as the time required for an environmental event to produce an effective change in physical action. It develops and compares three approaches: event-triggered preemptible skills, delay-aware asynchronous action chunking, and predictive control with uncertainty. Robot-RSI provides the initial execution infrastructure. Controlled dynamic perturbations will separate sensing, detection, reasoning, scheduling, and actuation delays, while jointly evaluating recovery, final success, and computational cost. The intended paper asks when execution-time adaptation yields benefits beyond faster inference or more frequent observation.

1. Research Problem and Standalone Paper Claim

A concrete failure. A robot reaches for a cup, but the cup moves during the reach. Producing the next instruction faster does not ensure that the old motion has stopped. A new instruction based on the earlier image may simply execute an outdated action sooner. Grasp slip requires a different response, and a revised user instruction may require a new goal. These events differ in urgency and in what constitutes successful correction.

Starting from Robot-RSI. Some existing motion skills already contain local feedback, and the Flexiv contact-descent routine can detect force internally and stop. These capabilities must be distinguished from task-level adaptation during execution. In the inspected sidecar, motion, state reads, and stop requests share a request lock, so an external observation or stop can wait for the current handler. This finding concerns the software request path, not independent hardware emergency-stop behavior. The research target is timely incorporation of new evidence into execution.

Definition. Let t_e denote the physical event time and t_a the time when a corrective action begins to take effect. End-to-end response latency is $L = t_a - t_e$. The evaluator records t_e without announcing it to the policy. Recovery time and final task success are separate endpoints: immediate stopping may shorten a response measure without completing the task. A text statement that the agent has responded is not an actuation measurement.

Central claim to test. Managing action validity and commitment under changing observations can improve disturbed-task success and response-tail latency at matched compute, beyond merely increasing observation frequency or reducing model latency. Event-triggered execution, asynchronous chunking, and predictive control provide three technical routes to this claim. Each has an appropriate matched baseline; they need not be combined to demonstrate an effect.

Prior work and hypotheses. Real-Time Chunking (RTC) already studies asynchronous action-chunk execution. The proposed increment concerns invalidation by environmental change, measured physical response, and its relationship to final success. H1: genuine preemption improves recovery at matched sensing frequency. H2: delay- and state-conditioned action commitment improves over fixed chunking at matched inference budget. H3: calibrated short-horizon prediction improves delayed control, while prediction errors can cause regression. The paper must identify these boundaries instead of assuming that asynchronous or predictive systems are inherently more responsive.

Primary method and scope. The main method will be event-triggered, preemptible execution in Approach A. Asynchronous chunking is a matched execution baseline and possible integration path; predictive control is an extension. The paper centers on whether observable changes invalidate actions and produce timely, effective recovery under matched sensing and compute.

2. Approach A: Event-Triggered, Preemptible Skills

Events and validity conditions. A skill declares observable preconditions: its target remains within a valid region, grasp state is unchanged, or the visible path remains compatible with the plan. A timestamped sensor stream feeds an event detector using change magnitude, confidence, and task phase. Small deviations may remain within the local controller’s responsibility. Larger changes trigger recovery or replanning, preventing every noisy image from interrupting stable motion.

Preemption protocol. Action packets carry a skill identifier, plan version, source-observation timestamp, expiry time, and cancellation marker. When an event invalidates a packet, the runtime revokes unexecuted commands and a single control owner stops or changes the motion and acknowledges the transition. A later model result based on an obsolete plan or observation must be revalidated before use. Cancelling a software wait does not stop the robot; logs must distinguish cancellation requested, accepted, and physically observed.

Recovery. Prefer local, verifiable responses such as slowing, holding, retracting briefly, or observing again. Invoke a slower Planner when the goal or program assumptions have changed. Debouncing, hysteresis, and cooldown reduce repeated triggers, but cooldown can also conceal important events and must be evaluated. Compare fixed thresholds with an event score learned on development data and frozen at test time. Ground-truth event times may support offline analyses but never drive the online detector.

Core comparison. Compare end-of-skill inspection, periodic inspection, and event-triggered execution with identical sensing rates. Measure false triggers, missed events, revoked commands, and stale actions that still execute. This separates better detection from actual preemptibility. Implement priority cancellation through the control owner or its queue, rather than allowing multiple threads to switch robot modes concurrently.

3. Approach B: Delay-Aware Asynchronous Action Chunking

The tradeoff. A continuous-action model predicts H future actions at once. Longer chunks reduce inference stalls but extend the influence of stale observations. Short chunks update more often, yet inference can outlast their execution and create gaps or discontinuities. The proposed method overlaps next-chunk inference with current execution and adapts the committed prefix to measured delay and scene change.

Alignment. With control interval Δt and estimated inference-plus-transfer delay $\hat{L}$, an initial commitment estimate is $p \approx \lceil \hat{L} / \Delta t \rceil$, refined by actions already executed and the device’s irrevocable buffer. A new chunk must join the actual execution state, not the state when inference was requested. Recent delay distributions support the estimate, with explicit behavior under unexpected tail latency.

Conditioning on change. A fixed prefix may be unnecessarily conservative in static scenes and too slow under target motion. Use target velocity, contact changes, and uncertainty to adjust the horizon and overlap. If the prefix becomes invalid, replace it at the cancellation boundaries supported by the device. Velocity or acceleration continuity constraints reduce abrupt transitions. The method must report any movement restrictions introduced by this commitment strategy.

Research increment. RTC is the main baseline. Compare fixed asynchronous chunks, delay-only adaptation, state-change-only adaptation, and their combination using the same action model and weights. This keeps the research centered on execution. If state-aware commitment does not improve final success or response tails, the conclusion should be limited to the measured continuity or utilization benefit.

4. Approach C: Predictive State Updates and Control

Predicting the state at actuation. Instead of using the latest target position directly, estimate the state when an action will take effect. Inputs include observation history, proprioception, candidate actions, and measured delay. A short-horizon model predicts future state and uncertainty. Receding-horizon optimization selects an action sequence, executes a prefix, and corrects predictions using new observations. Constant-velocity extrapolation and simple dynamics are explicit baselines before introducing a learned action-conditioned model.

Objective. Balance expected task cost, uncertainty, and control changes:

$$\min_{a_{t:t+H}} \mathbb{E}[C_{\text{task}}] + \lambda_U C_{\text{uncertainty}} + \lambda_A C_{\text{action}}.$$

High uncertainty may shorten the horizon, request another observation, or favor a conservative local action. The predictor uses permitted observations only. Simulator future states can measure offline prediction errors but are never supplied to the deployable planner.

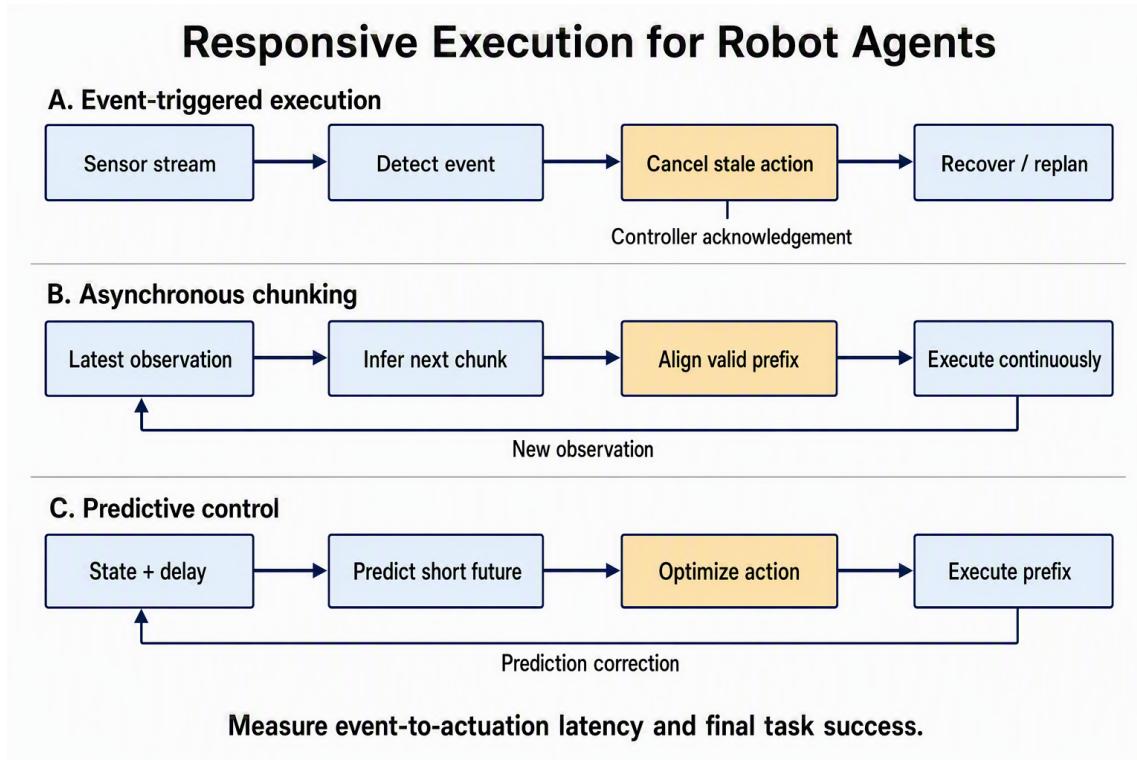


Figure 1: Independent pipelines for event-triggered execution, asynchronous chunking, and prediction. GPT-image-2 conceptual diagram. The evaluation target is physical correction and eventual task completion, not the time at which the model finishes generating a response.

Mechanism tests. Compare smooth target motion, sudden direction changes, brief occlusion, and contact-condition changes. Prediction may help under smooth motion and fail under abrupt changes; uncertainty calibration and observation correction should explain the boundary. Report both prediction accuracy and control performance. More accurate images or states do not automatically imply better actions, and excessive conservatism can reduce completion. Hardware timestamps must be aligned before making millisecond-scale claims.

5. Experimental Design: Dynamic Tasks and Delay Interventions

Perturbation panel. Extend LIBERO short tasks with four task-appropriate event classes: target movement, grasp/contact change, visible path-condition change, and instruction revision. Events occur before completion of the relevant action and leave a feasible task. For example, arbitrary teleportation of an already grasped target is not used as a natural-motion perturbation. An independent evaluator injects events, and policies discover them through declared sensing or instruction channels.

Proposed scale. Select 24 task–event pairs, covering each event class across multiple manipulation tasks. Three disturbance strengths, four delay conditions, and five seeds give 1,440 episodes per method. Pairs sharing an underlying task are not statistically independent. Separate development and validation panels determine solvability, magnitude, and timing before the test list is frozen. Adjust sample requirements using development variance rather than observed test effects.

Delay interventions. An initial design adds 0, 100, 300, or 600 ms of inference delay while recording actual total latency. Separate experiments add delay to sensing transfer or action submission; these are distinct interventions. A tail-latency panel compares distributions with similar means and different p95 values. If baseline inference already greatly exceeds the proposed bins, revise them against measured control and task timescales on validation, then freeze them.

Method	Update and execution	Mechanism isolated
Synchronous skill baseline	Observe after skill completion	Existing task-level feedback
Periodic inspection	Matched sensing, fixed update period	Value of observation frequency
Event preemption	Validity checking and acknowledged cancellation	Interruption and recovery
Short synchronous chunks	Frequent inference before execution	Resource cost of updates
RTC baseline	Asynchronous fixed-model chunks	Continuity under inference delay
State-aware chunking	Adaptive commitment and invalidation	Joint response to delay and change
Predictive control	Delay compensation and correction	Value of anticipating future state

Controlling confounds. Event methods share perception models and rates; chunking methods share action-model weights; predictive methods are compared under matched compute and movement limits. Record sampling rates, queue length, inference calls, and cumulative compute. Giving one method more GPUs or denser sensing does not establish a scheduling contribution. Report matched-budget comparisons and success–latency–cost tradeoffs where exact matching is impossible.

Benchmark boundaries. Report dynamic extensions separately from the standard 120-task short panel. The latter uses task-level pass@10 with spatial/object/goal and task/object/swap/lan breakdowns. Dynamic main tables report success on fixed perturbation instances and latency distributions. Exact manifests bind tasks, seeds, event trajectories, trigger rules, latency samples, and method versions. Preserve evidence and successful-seed resume protection. Cross-method trials are independent comparisons, while an unchanged method does not rerun an already successful task/seed configuration.

6. Planned Experiments: From Events to Recovery

Timing decomposition. Record physical event time t_e , the first sample containing the event t_s , detection t_d , corrective decision readiness t_p , controller acceptance t_c , and physical action change t_a . Report $t_s - t_e$, $t_d - t_s$, $t_p - t_d$, $t_c - t_p$, and $t_a - t_c$ where defined. A continuously updating method may have no discrete detection event; compare common measurable stages without inventing timestamps.

Defining effective response. Prespecify criteria by event: motion toward the changed target, grasp maintenance or recovery after slip, slowing/stopping/rerouting after obstruction, or ceasing actions that contradict a revised instruction. Validation freezes these rules, and the evaluator applies them offline. Observable physical change establishes response; a Planner declaration does not. Final task success remains an independent harness verdict.

Experiment A: event-triggered mechanisms. At matched sensing rates, ablate preemption, expiry checks, and learned versus fixed event thresholds. Measure false triggers, missed events, failed acknowledgements, and stale commands. Introduce noise and occlusion to test whether hysteresis generalizes. Accurate detection without physical correction points to the execution path; fast stopping without successful recovery supports only a narrower claim.

Experiment B: asynchronous execution and tails. Vary the mean, variance, and spikes of inference latency, then examine chunk length, committed prefix, state-change threshold, and transition smoothing. Primary measures are disturbed-task success, response p95, and continuity; report p50, p99, and control gaps as secondary measures. Strong p99 claims require enough events. Measure velocity and acceleration changes so that aggressive motions cannot masquerade as an unqualified responsiveness gain.

Experiment C: whether prediction improves control. Remove delay compensation, uncertainty penalties, and observation correction separately. Compare constant-velocity and learned prediction for smooth motion and sudden turns. Analyze relationships between prediction error and task failure. Any future-state upper-bound analysis remains offline, in a separate table, with no future ground truth exposed to the evaluated agent.

Experiment D: responsiveness versus cost. At fixed call or compute budgets, compare success, response p95, tokens, and wall time. Event triggering may save unnecessary model calls in static scenes. Asynchrony may improve utilization while increasing cumulative compute. Prediction may be more accurate but too slow to help. Evaluate end-to-end cost rather than only reduced time waiting for a language model.

Possible observation	Appropriate conclusion
Faster response and higher disturbed-task success	Supports effective responsiveness after matching sensing and compute.
Faster stopping but slower recovery or more failures	Improves the stop path, not necessarily dynamic manipulation.
Smooth chunks still execute stale actions after abrupt changes	Continuity improves while validity management remains a bottleneck.
Prediction helps smooth motion but hurts sudden turns	State the motion boundary and analyze correction and uncertainty.
Frequent periodic checks perform equally well	The event method's distinct benefit may be computational savings.

7. Statistical Protocol, Risks, and Paper Contribution

Statistics. Use paired bootstrap intervals clustered by underlying task, sharing event trajectories and latency samples across methods. Preregister final disturbed-task success and response p95; detection, recovery, and compute are explanatory metrics. Episodes that never respond cannot silently disappear from latency statistics. Report nonresponse rate, timeout threshold, and conditional latency quantiles; use censoring-aware analysis where appropriate. Otherwise, a method responding only on easy cases could appear artificially fast.

Physical execution evidence. A submitted command may wait while the robot executes an older buffer. Validate response through proprioception, controller acknowledgements, and synchronized timestamps. Quantify inter-device clock error. Keep simulator physical time separate from wall time, especially in accelerated simulation. Record frame production, arrival, and model-consumption times separately to expose observation staleness.

Principal risks. Event detectors can overfit task-specific signals, requiring task-family holdouts. Repeated interruptions can destabilize otherwise effective local control, requiring explicit cancellation boundaries and smooth transitions. Predictive models can fail abruptly, requiring calibration and regression analysis. The three approaches require different model or execution interfaces, so use matched baselines for each rather than forcing an unfair single ranking.

Outcome and efficiency reporting. Only simulator or independent-harness verdicts count as task success. Provider, transport, image, resource, and interrupted records are excluded from valid-verdict task denominators and reported separately. Also report implementation failures, nonresponses, protection events, and incomplete coverage. Log prompt/completion/total tokens, VLM and unmetered calls, wall time, and perception/action/recovery time. Preserve videos, trajectories, failures, and diagnostic logs. Learned models must be evaluated on matched and held-out tasks; lower training loss is not proof of improved response.

Expected contribution. This paper should establish an execution-time account of responsiveness: when event-triggered validity checks, adaptive action commitment, or short-horizon prediction can improve behavior under environmental change. Its methodological contribution is a comparison with matched sensing and compute; its evaluation contribution is the separation of event time, physical correction, recovery, and final completion. The strongest supported claim must depend on actual closed-loop behavior.

Independence. The study can use fixed skills or a frozen VLA and does not depend on learning a new force controller or persistent GPT review. Candidate methods receive independent ablations under the common question. Task counts, perturbations, and delay bins are proposed experimental settings to freeze after validation, not completed measurements or guaranteed gains.

References

- [1] Real-Time Execution of Action Chunking Flow Policies. 2025. <https://arxiv.org/abs/2506.07339>
- [2] Diffusion Policy: Visuomotor Policy Learning via Action Diffusion. 2023. <https://arxiv.org/abs/2303.04137>
- [3] π_0 : A Vision-Language-Action Flow Model for General Robot Control. 2024. <https://arxiv.org/abs/2410.24164>
- [4] Code as Policies: Language Model Programs for Embodied Control. 2022. <https://arxiv.org/abs/2209.07753>
- [5] Noematrix Team. RoboRSI research preview. 2026. <https://lab.noematrix.ai/blog/2-roborsi-research-preview/>